



**Universidad
Gerardo Barrios**

Nombre del proyecto:

ReceiptIA

Módulo:

Desarrollo de Aplicaciones con IA

Integrantes:

Jonathan Elías Gámez Larin

Wilber José Jiménez Ramírez

David Arnoldo Hernández Gómez

Fecha de entrega:

15/08/26

Instrucción del sistema de observación

```
request_id = str(uuid.uuid4())
inicio = time.perf_counter()

try:
    response = await call_next(request)

    status_code = response.status_code

    if status_code >= 500:
        error_type = "internal_error"
    elif status_code >= 400:
        error_type = "client_error"
    else:
        error_type = "none"

    return response

except Exception:
    status_code = 500
    error_type = "internal_error"
    raise

finally:
    duracion_ms = (time.perf_counter() - inicio) * 1000

    logger.info(
        "request_id=%s | route=%s | status=%s | "
        "duration_ms=%.2f | ai_model=%s | error_type=%s",
        request_id,
        request.url.path,
        status_code,
        duracion_ms,
        AI_MODEL,
        error_type,
    )
```

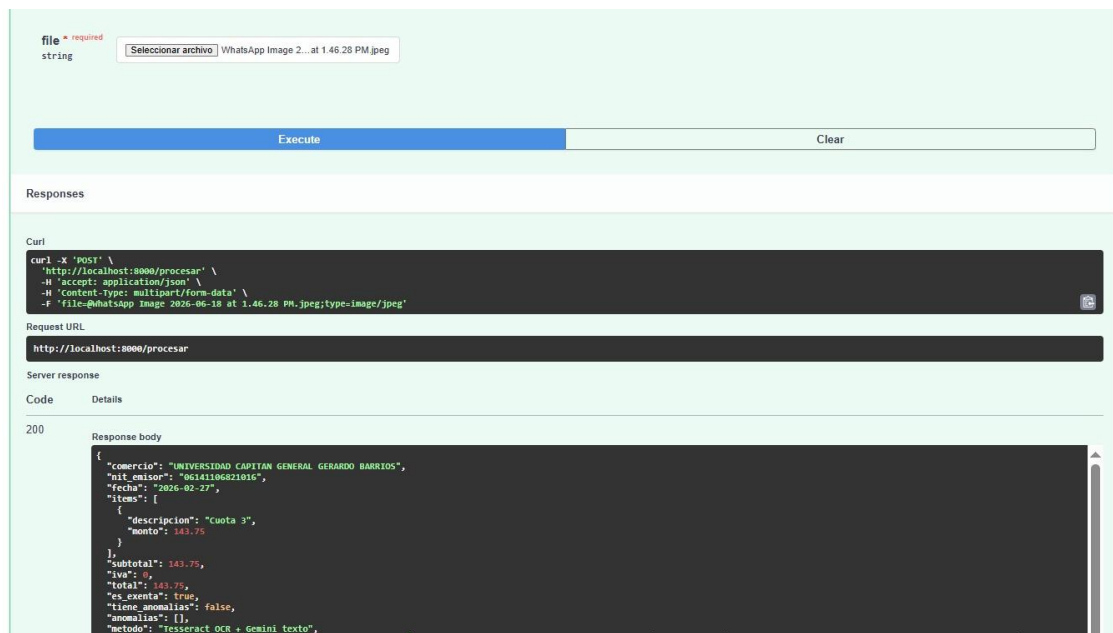
Se muestra el middleware que será el encargado de registrar información de cada solicitud incluyendo un identificador único "request_id" el código de estado HTTP, esta implementación permite monitorear el comportamiento del endpoint y facilita la identificación de solicitudes exitosas o fallas, además de llegar a medir el tiempo de respuesta y detectar errores durante el procesamiento.

Manejo de facturas no validas

```
if len(facturas_validas) == 0:
    return {
        "facturas": facturas_invalidas,
        "metodo": "Tesseract OCR + Gemini texto en lote",
        "cantidad": len(facturas_invalidas)
    }
```

Se muestra la validación del resultado del procesamiento, verificación si no existen facturas validas y devolviendo las facturas identificadas como invalidas junto con el método utilizado y las cantidades detectadas.

Prueba del endpoint principal /procesar



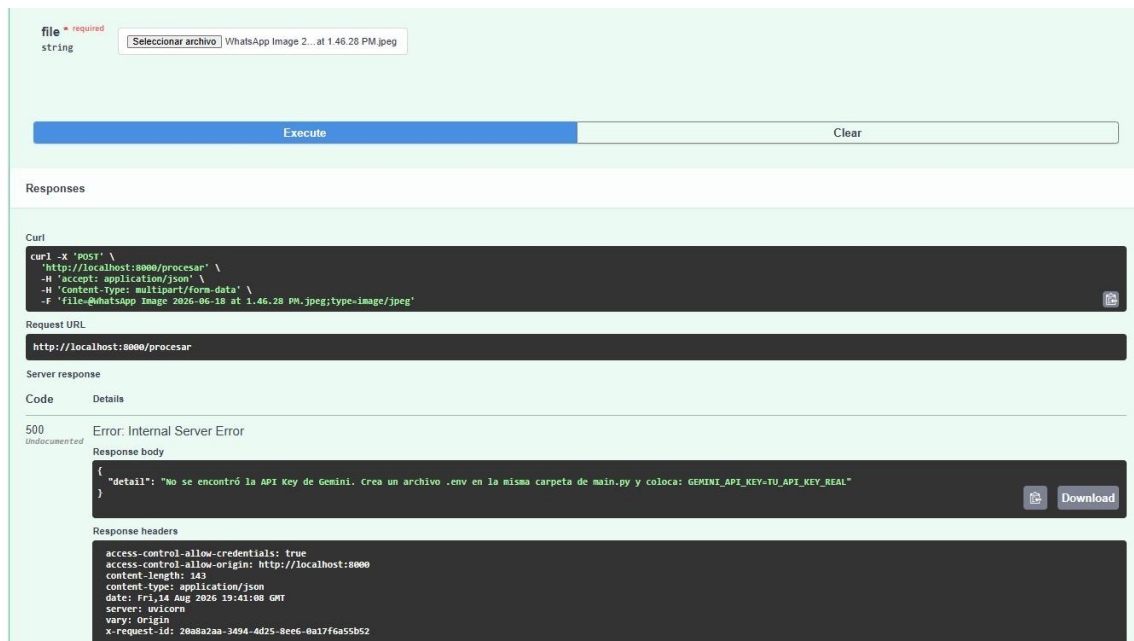
Se muestra la ejecución del endpoint principal /procesar mediante Swagger UI enviando la imagen de una factura el archivo responde con el código HTTP 200, indicando que la solicitud fue procesada exitosamente y así mismo devolviendo los datos extraídos y analizados, la prueba confirma el funcionamiento correcto del flujo principal mediante el procesamiento OCR y el análisis con IA.

Registro de una solicitud exitosa

```
PS C:\Users\Wilber JJR\OneDrive - Universidad Gerardo Barrios\Escritorio\ReceiptIA-main\Backend> dock
er run --env-file .env -p 8000:8000 receiptia
2026-08-14 19:51:39,027 | INFO | OCR finalizado | caracteres_extraidos=2170
2026-08-14 19:51:39,027 | INFO | Enviando información procesada a Gemini
2026-08-14 19:51:39,077 | INFO | AFC is enabled with max remote calls: 10.
2026-08-14 19:51:39,077 | WARNING | Direct use of automatic function calling (AFC) in Models.generate
_content is not recommended. Instead, we recommend to use AFC in Chat.send_message. Similarly, direct
use of AFC in Models.generate_content_stream is not recommended. Instead, we recommend to use AFC in
Chat.send_message_stream.
2026-08-14 19:51:44,741 | INFO | HTTP Request: POST https://generativelanguage.googleapis.com/v1beta/
models/gemini-2.5-flash:generateContent "HTTP/1.1 200 OK"
2026-08-14 19:51:44,751 | INFO | Gemini respondió correctamente
2026-08-14 19:51:44,753 | INFO | Procesamiento individual finalizado correctamente
2026-08-14 19:51:44,758 | INFO | request_id=841e62cd-f434-4ed6-9849-7ff4fe4def11 | route=/procesar |
status=200 | duration_ms=7622.14 | ai_model=gemini-2.5-flash | error_type=none
INFO: 172.17.0.1:45748 - "POST /procesar HTTP/1.1" 200 OK
```

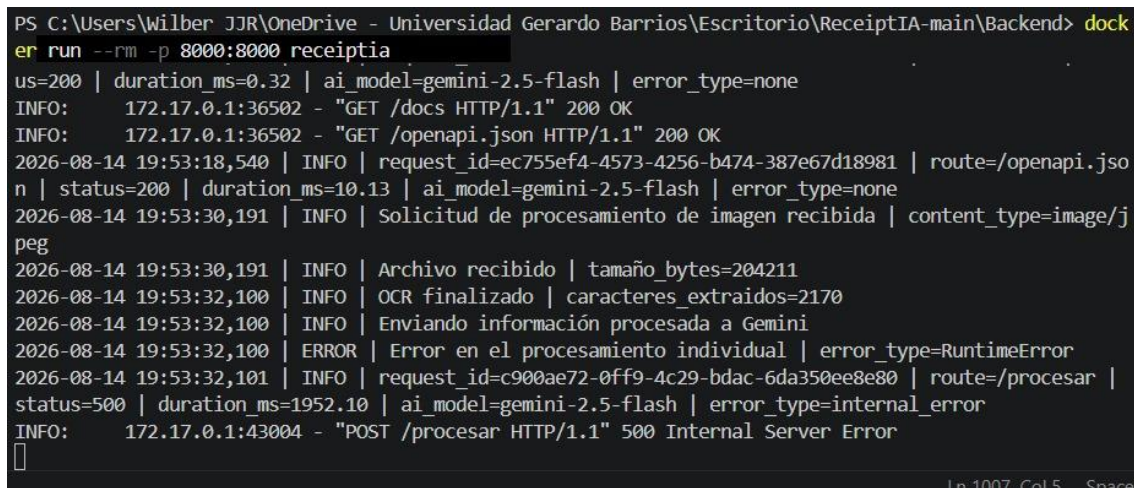
Se muestra el registro generado durante el procesamiento de una solicitud al endpoint /procesar el sistema registra un "request_id" la ruta, el código de estado HTTP, la duración del procesamiento, el modelo de IA usada y el tipo de error, también confirma que la solicitud fue procesada correctamente con un HTTP 200 y con una duración de 7622.14 ms y sin errores.

Manejo de error controlado



Se observa una prueba del endpoint `/procesar` en Swagger UI en la que el servidor devuelve un código HTTP 500 (Internal Server Error) debido a la ausencia de la clave de API en Gemini, el sistema identifica y comunica un fallo durante el procesamiento permitiendo registrar y controlar errores internos sin exponer la información sensible.

Registro de error y observabilidad



Se muestra el registro generado ante un fallo durante el procesamiento del endpoint `/procesar` la evidencia confirma que el error es identificado y registrado mediante los mecanismos de observabilidad implementados, la solicitud finaliza con el HTTP 500 y `error_type=internal_error` permitiendo diferenciar una falla interna de una solicitud procesada correctamente.

Middleware de observabilidad

```
@app.middleware("http")
async def observability_middleware(request: Request, call_next):
    """Registra una línea estructurada por solicitud.

    Campos mínimos de Semana 5: request_id, ruta, estado, duración y versión
    del componente IA. El identificador también se devuelve en X-Request-ID.
    """
    request_id = request.headers.get("X-Request-ID") or str(uuid.uuid4())
    token = _request_id_ctx.set(request_id)
    started = time.perf_counter()
    status_code = 500
    event = "request_error"
    error_type = None
```

Se muestra que la implementación de middleware de observabilidad permite realizar un seguimiento de las solicitudes y distinguir entre diferentes tipos de resultados, facilitando el monitoreo y diagnóstico del comportamiento del sistema sin almacenar directamente el contenido sensible de las facturas.

Prueba de rendimiento con 20 solicitudes

```
PS C:\Users\Wilber JJR\OneDrive - Universidad Gerardo Barrios\Escritorio\ReceiptIA-main> python bench
mark.py
=====
BENCHMARK RECEIPTIA - SEMANA 5
=====
Endpoint: http://127.0.0.1:8000/procesar
Imagen: factura_prueba.jpg
Solicitudes: 20
=====

Solicitud 1/20...
Estado: 200 | Duración: 17725.83 ms | Request ID: a72f3de2-6fda-4d58-97f3-a6d9d64b6d43

Solicitud 2/20...
Estado: 200 | Duración: 9023.53 ms | Request ID: 782c7cd7-a755-4f1f-8fed-3daeb37a8941

Solicitud 3/20...
Estado: 200 | Duración: 9917.13 ms | Request ID: e891ce34-cc60-4d52-96e2-1baefae38e

Solicitud 4/20...
ERROR HTTP 500 | Duración: 3490.93 ms | Request ID: c05dfb1d-2e13-46f5-bb8a-bf66e0b0942f

Solicitud 5/20...
Estado: 200 | Duración: 13501.07 ms | Request ID: a589b3e2-32b3-47ea-ba4f-6bcd555ce54e

Solicitud 6/20...
Estado: 200 | Duración: 8635.31 ms | Request ID: 5999cb07-acco-4361-83cf-2bf8a6bb9f10

Solicitud 7/20...
Estado: 200 | Duración: 9134.52 ms | Request ID: 6cf2e176-502d-4705-adea-97f9d05343f5

Solicitud 8/20...
Estado: 200 | Duración: 10636.91 ms | Request ID: ddc265c5-83d8-4042-96b3-cbfe47e212f0

Solicitud 9/20...
Estado: 200 | Duración: 9594.54 ms | Request ID: 613f75fa-f6ce-48d3-9a5b-2058bca27768

Solicitud 10/20...
Estado: 200 | Duración: 10997.85 ms | Request ID: 07068d5d-c04f-4c20-9e6c-1b460e8bb709

Solicitud 11/20...
Estado: 200 | Duración: 11147.78 ms | Request ID: c10fd81a-0b49-47f5-90cc-48f164ab9c6c

Solicitud 12/20...
Estado: 200 | Duración: 10889.94 ms | Request ID: 8c41e392-00da-4cca-8230-a96b6f070917

Solicitud 13/20...
Estado: 200 | Duración: 10727.23 ms | Request ID: 126418e2-ea7a-4bb6-8345-a14a05d75776
In 284 Col 25
```



```

Solicitud 14/20...
  Estado: 200 | Duración: 11406.85 ms | Request ID: 14a420b5-beb7-482f-a76a-09df47544c7b

Solicitud 15/20...
  ERROR HTTP 500 | Duración: 2160.85 ms | Request ID: f8749011-6c5e-43e9-9efd-ce749ef4be9a

Solicitud 16/20...
  Estado: 200 | Duración: 11188.61 ms | Request ID: 26f5ea6c-bed8-4c5d-b402-6041fe932ceb

Solicitud 17/20...
  ERROR HTTP 500 | Duración: 2099.67 ms | Request ID: 84cf2431-9bd7-4615-b003-ff293bd8a98c

Solicitud 18/20...
  Estado: 200 | Duración: 12312.03 ms | Request ID: 31899903-9bda-4d28-8ca1-e2fa6db740aa

Solicitud 19/20...
  Estado: 200 | Duración: 10829.39 ms | Request ID: af57c0bf-5b63-4092-bc82-0ee5ded4df68

Solicitud 20/20...
  ERROR HTTP 500 | Duración: 2241.19 ms | Request ID: ad3ac6af-6417-4bf4-8029-4c449b15195a

```

La prueba permite establecer una línea base del rendimiento del sistema, de las 20 solicitudes realizadas, se observan respuestas exitosas como HTTP 200 y algunos errores HTTP 500 además de variar el tiempo de procesamiento.

Resultados de la prueba de rendimiento

```

=====
RESULTADOS DEL BENCHMARK
=====
Solicitudes totales : 20
Solicitudes exitosas: 16
Errores              : 4
Tasa de error        : 20.00%
p50                  : 10682.07 ms
p95                  : 13712.31 ms
Máximo               : 17725.83 ms
=====

Archivos generados:
- benchmark_resultados.csv
- benchmark_resumen.txt
PS C:\Users\Wilber JJR\OneDrive - Universidad Gerardo Barrios\Escritorio\ReceiptIA-main>

```

Los resultados de las 20 solicitudes realizadas presentan 16 solicitudes exitosas y 4 errores, equivalente a una tasa de error del 20 %. Las métricas de latencia obtenidas fueron un p50 de 10.68 segundos, un p95 de 13.71 segundos y un tiempo máximo de 17.73 segundos. Estos resultados permiten establecer una línea base de rendimiento e identificar oportunidades de mejora para futuras optimizaciones.

Archivo de resultados de rendimiento

```
benchmark_resumen.txt X
benchmark_resumen.txt
1  BENCHMARK RECEIPTIA - SEMANA 5
2  =====
3
4  Endpoint: http://127.0.0.1:8000/procesar
5  Solicitudes: 20
6  Exitosas: 16
7  Errores: 4
8  Tasa de error: 20.00%
9  p50: 10682.07 ms
10 p95: 13712.31 ms
11 Máximo: 17725.83 ms
12
```

Se muestra el archivo `benchmark_resumen.txt`, generado automáticamente después de ejecutar la prueba de rendimiento. Este archivo contiene el resumen de las métricas obtenidas en las 20 solicitudes, incluyendo solicitudes exitosas, errores, tasa de error, p50, p95 y tiempo máximo.

Plan de escalabilidad

El plan debe responder:

- ¿Qué crecimiento se está considerando y cuál es la restricción observada?

Se considera un crecimiento progresivo en la cantidad de usuarios y solicitudes que procesan facturas de forma simultánea, la principal restricción observadas en las pruebas es la latencia de procesamiento con un p50 de 10.68 segundos y un p95 de 13.71 segundos, además de una tasa de error del 20% en as 20 solicitudes realizadas

- ¿Qué mejora debe realizarse primero?

La primera mejora debe ser en reducir la latencia y estabilizar el procesamiento de las solicitudes, se propone optimizar el flujo de OCR y comunicación con Gemini, también revisar las causas de los errores HTTP 500 observados durante las pruebas

- ¿Cuándo convendría usar caché, workers, colas o más instancias?

- Cache: cuando existe consultas o documentos que se pueden reutilizar resultados sin necesidad de procesarlos de nuevo.
- Workers: Cuando aumente la cantidad de solicitudes simultaneas y sea necesario procesar varias facturas.
- Colas: Cuando el volumen de solicitudes supere la capacidad de procesamiento inmediato, permitiría administrar las solicitudes pendientes sin bloquear al usuario.

- Mas instancias: Cuando una sola instancia del backend ya no pueda mantener tiempos de respuesta aceptables ante el aumento de usuarios o solicitudes constantes.

- ¿Qué impacto tendría en memoria, datos, privacidad y costo?

El aumento de workers o instancias incrementaría el consumo de memoria y CPU mientras que una mayor cantidad de procesamiento podría aumentar los costos de infraestructura y del uso de servicios de IA, los documentos procesados contienen información potencialmente sensible por lo que se debe evitar almacenar innecesariamente imágenes, OCR o datos personales y mantener las credenciales de API protegidas mediante variables de entorno.

- ¿Qué indicador permitiría decidir cuándo escalar?

El principal cuello de botella observado es la latencia del flujo de procesamiento de facturas, que incluye las etapas de OCR y comunicación con el modelo Gemini. El p95 de 13.71 segundos indica que las solicitudes de mayor duración presentan una latencia considerablemente superior al comportamiento mediano (p50 de 10.68 segundos). Por ello, la primera optimización debería enfocarse en reducir el tiempo total del pipeline y estabilizar los errores HTTP 500.

Link del repositorio: <https://github.com/DavidHernandezd/ReceiptIA.git>